MACHINE LEARNING • BAB 6

# KNN dan Naive Bayes

6.16 Implementasi Python: Praktikum 6.1 sampai 6.7

Ahmad Dandi Subhani

202343500126 • Kelas R7B

PENDAHULUAN

# Tujuan Praktikum

- Memahami cara kerja **KNN** dan **Naive Bayes** pada data nyata
- Membuktikan kenapa **scaling wajib untuk KNN** tapi tidak untuk Naive Bayes
- Menjawab: kapan KNN lebih baik, kapan Naive Bayes lebih baik, dan kenapa?

**Dataset:** Wine Recognition • 178 sampel, 13 fitur kimia
**Kelas:** 3 kultivar anggur (59 / 71 / 48) • Sumber: sklearn.datasets

PRAKTIKUM 6.1

# Persiapan Library

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

- Tiga kelompok: **komputasi dan visualisasi**, **data dan preprocessing**, **model dan evaluasi**

PRAKTIKUM 6.2

# Import Dataset

```
data = load_wine(as_frame=True)
X, y = data.data, data.target
print("Ukuran data:", X.shape)
print("Distribusi kelas:", np.bincount(y))
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42, stratify=y)
```

```
Ukuran data: (178, 13)
Distribusi kelas: [59 71 48]
```

- 80% latih (142), 20% uji (36) • **stratify** menjaga proporsi kelas • **random_state=42** supaya bisa diulang

PRAKTIKUM 6.3

# Preprocessing: Scaling untuk KNN

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Catatan: Naive Bayes tidak memerlukan scaling,
# namun KNN sangat sensitif terhadap skala fitur.
```

- KNN menghitung **jarak**: fitur berentang besar akan mendominasi tanpa scaling
- **fit** hanya di data latih, lalu **transform** ke data uji (no data leakage)
- Naive Bayes bekerja dengan **distribusi probabilitas**, bukan jarak

PRAKTIKUM 6.4

# Training Model

```
knn = KNeighborsClassifier(n_neighbors=5,
    weights="distance")
knn.fit(X_train_scaled, y_train)

nb = GaussianNB()
nb.fit(X_train, y_train)
```

- **KNN**: k=5, weights="distance" (tetangga dekat bobotnya lebih besar), latih di data **scaled**
- **GaussianNB**: tanpa parameter, latih di data **asli** (asumsi distribusi Gaussian per fitur)

PRAKTIKUM 6.5

# Evaluasi

```
acc_knn = accuracy_score(y_test, knn.predict(X_test_scaled))
acc_nb = accuracy_score(y_test, nb.predict(X_test))
print(f"Akurasi KNN (k=5, weighted): {acc_knn:.3f}")
print(f"Akurasi Gaussian Naive Bayes: {acc_nb:.3f}")
```

```
Akurasi KNN (k=5, weighted): 0.972
Akurasi Gaussian Naive Bayes: 0.972
```

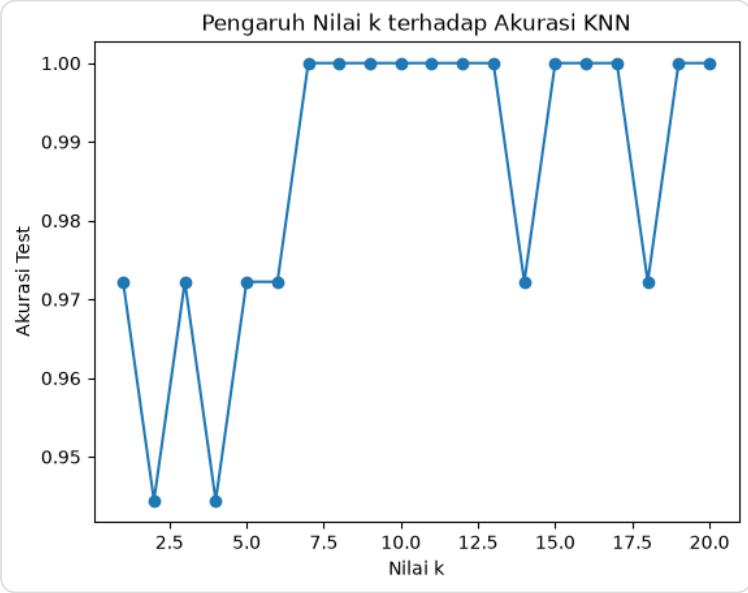
- **Seri 0,972**: 35 dari 36 data uji benar • dua algoritma beda cara, sama-sama tepat

PRAKTIKUM 6.6

# Visualisasi: Pengaruh Nilai k

```
akurasi_k = []
nilai_k = range(1, 21)
for k in nilai_k:
    model = KNeighborsClassifier(
        n_neighbors=k).fit(
        X_train_scaled, y_train)
    akurasi_k.append(accuracy_score(
        y_test,
        model.predict(X_test_scaled)))

plt.plot(nilai_k, akurasi_k, marker='o')
plt.xlabel("Nilai k")
plt.ylabel("Akurasi Test")
plt.title("Pengaruh Nilai k terhadap
Akurasi KNN")
plt.show()
```




- k kecil (1-6): **fluktuatif** 0,944-0,972, sensitif noise
- **k=7 optimal**: akurasi 1,000 dan stabil

PRAKTIKUM 6.7

# Klasifikasi Teks Spam

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

teks = ["dapatkan diskon gratis sekarang", "rapat tim jam 10 pagi",
        "menangkan hadiah gratis klik disini", ...]   # 6 kalimat
label = [1, 0, 1, 0, 1, 0]  # 1 = spam, 0 = bukan spam
vectorizer = CountVectorizer()
X_teks = vectorizer.fit_transform(teks)
model = MultinomialNB(alpha=1.0)  # Laplace Smoothing
model.fit(X_teks, label)
teks_baru = ["diskon gratis hari ini"]
prediksi = model.predict(vectorizer.transform(teks_baru))
```

```
Prediksi: Spam   (benar: kata "diskon" dan "gratis" dominan di kelas spam)
```

# Apa Artinya?

- KNN dan Naive Bayes **seri 0,972**: pola kelas data Wine memang terpisah dengan baik
- **Scaling wajib untuk KNN**, tidak berpengaruh untuk Naive Bayes
- **k=7** adalah nilai optimal pertama (akurasi 1,000, stabil)
- Pemilihan k yang cermat idealnya lewat **cross-validation**, bukan satu split

# Latihan 1: KNN vs Naive Bayes + Waktu

- **Dataset:** Breast Cancer Wisconsin (569 sampel, 30 fitur), KNN dengan scaling
- **Akurasi:** KNN **0,9561** vs GaussianNB **0,9386** - KNN menang tipis
- **Waktu training:** KNN sekitar 1 ms (lazy learner, cuma simpan data) vs NB beberapa ms
- **Waktu prediksi:** KNN puluhan-ratusan ms (hitung jarak ke 455 data latih) vs NB di bawah 2 ms
- **Kesimpulan:** KNN menang akurasi tapi bayar mahal di waktu prediksi

# Latihan 2: MultinomialNB untuk Teks

- **Data:** 16 kalimat Bahasa Indonesia (8 spam, 8 bukan spam), split 75:25 + stratify
- **Akurasi uji: 0,7500** (3 dari 4 benar, wajar untuk data sekecil ini)
- Kalimat baru: "gratis ongkir hari ini" → **Spam**, "rapat koordinasi besok pagi" → **Bukan Spam** (keduanya benar)
- **Kesimpulan:** MultinomialNB cocok untuk bag-of-words; akurasi naik dengan data lebih besar

KESIMPULAN

# Tiga Poin Kunci

- **Scaling menentukan nasib KNN**, tapi tidak relevan untuk Naive Bayes
- **Tidak ada algoritma yang selalu menang**: seri di Wine (0,972), KNN unggul di Breast Cancer (0,9561 vs 0,9386)
- **Kecepatan vs akurasi**: KNN akurat tapi prediksinya lambat; Naive Bayes sedikit kalah akurat tapi kilat dan ringan

BAB 6 • KNN DAN NAIVE BAYES

# Terima Kasih

Ahmad Dandi Subhani

202343500126 • Kelas R7B